React 컴포넌트 설계 문서

문서 정보

- 프로젝트명: (SK Shieldus Rookies 내부 협업 메신저)
- 작성자: SK쉴더스 루키즈 4기 개발 2팀
- 작성일: 2025-09-16
- 버전: v1.0

---

1. Login

1.1 React 컴포넌트 개요

컴포넌트명

LoginForm

목적 및 역할

- 사용자의 인증을 위한 진입점 역할을 하는 컴포넌트
- 사용자가 이메일과 비밀번호를 입력하여 서비스에 로그인할 수 있도록 하는 UI를 제공하며, 인증 요청을 백엔드 API로 전송하고 결과를 처리하는 역할을 담당

주요 기능 및 요구사항:

- 사용자 인증 처리 : 입력받은 이메일과 비밀번호를 사용하여 백엔드 API 통신을 통해 사용자 인증을 시도합니다.
- 폼 데이터 관리 : 이메일과 비밀번호 입력 필드의 상태를 관리하고, 사용자의 입력에 따라 동적으로 업데이트합니다.
- 상태 관리 (Redux) : 로그인 성공 시 Redux store에 사용자 데이터를 저장하여 애플리케이션 전반에서 사용자의 로그인 상태를 전역적으로 관리합니다.
- 네비게이션 : 로그인이 성공하면 사용자를 권한에 따라 맞는 페이지(HomePage.tsx)로 리디렉션합니다.
- 연결 기능 : 비밀번호 찾기(FindPassword.tsx) 및 회원가입(RegisterForm.tsx) 페이지로 이동하는 링크를 제공하여 사용자 경험을 개선합니다.
- 로그인 정보 기억 : localStorage를 사용하여 '로그인 정보 기억' 기능 상태를 저장하고, 다음 접속 시에도 이 설정이 유지되도록 합니다.

1.2. 컴포넌트 구조 설계

컴포넌트 분류

- 타입: [✔] 페이지 컴포넌트 [ ] 레이아웃 컴포넌트 [] 기능 컴포넌트 [ ] 공통 컴포넌트
- 복잡도: [ ] 단순 (50줄 이하) [ ] 보통 (50-100줄) [✔] 복합 (100줄 이상)
- 상태 관리: [✔] 로컬 상태만 [✔] Redux 연결 [ ] 상태 없음

파일 구조

```
src/
├── components/
│   ├── auth/
│   │   ├── FIndPassword.tsx
│   │   ├── LoginForm.tsx
│   │   ├── RegisterForm.tsx
│   │   ├── RegisterSuccess.tsx
│   │   └── ResetPassword.tsx
├── store/
│   ├──slices/
│   │   └── authSlice.ts
│   ├── index.ts
    └── hooks.ts
```

1.3 상태 관리 설계

로컬상태

상태명	타입	초기값	용도
formData	{ email: string, password: string }	{ email: "", password: "" }	사용자가 입력하는 이메일과 비밀번호 관리
remember	boolean	localStorage.getItem ("rememberMe") === "1"	체크박스의 상태를 관리하며, 로컬 스토리지에 값을 저장.

Redux상태

액션타입	Payload	설명
setUserData	object	로그인 성공 후 백엔드로부터 받은 사용자

		데이터를 전역 상태에 저장
logout	없음	사용자 인증 상태를 초기화하여 로그아웃 처리
initializeAuth	없음	애플리케이션 시작 시 로컬 또는 세션 스토리지에 저장된 토큰을 확인하여 인증 상태를 초기화
setEmail	string	비밀번호 찾기 과정에서 사용자가 입력한 이메일을 임시로 저장
setVerifyCode	string	비밀번호 찾기 과정에서 사용자가 입력한 인증 코드를 저장
setNewPassword	string	새로운 비밀번호 설정 과정에서 사용자가 입력한 새 비밀번호를 저장
resetAuthState	없음	비밀번호 찾기와 같은 특정 인증 관련 프로세스가 완료된 후, 해당 프로세스에 사용된 상태들을 초기화

1.4 구현 세부사항

주요 컴포넌트 목록

컴포넌트명	위치	역할	재사용성
FindPassword	/components/auth	비밀번호찾기	낮음
LoginForm	/components/auth	로그인	낮음
RegisterForm	/components/auth	회원가입	낮음
RegisterSuccess	/components/auth	페이지표시	낮음
ResetPassword	/components/auth	비밀번호찾기	낮음

API 연동

API 엔드포인트	메서드	용도	호출시점

/api/auth/password /reset?email=${encodeURIComponent(email)}	POST	비밀번호 찾기 위한 이메일 인증	버튼 선택
/api/auth/password/verify	POST	비밀번호 찾기 위한 이메일 인증	버튼 선택
/api/auth/password/update	POST	비밀번호 재설정	버튼선택
/api/auth/login	POST	로그인	버튼선택
/api/users/me	GET	로그인 시 권한,상태 가져오기	버튼선택
/api/auth/logout	POST	로그아웃	버튼선택
/api/users/signup	POST	회원가입	버튼선택
/api/auth/password /reset?email=${encodeURIComponent(email)}	POST	비밀번호 찾기 위한 이메일 인증	버튼 선택
/api/auth/password/verify	POST	비밀번호 찾기 위한 이메일 인증	버튼 선택

1.5 성능 최적화

- Redux를 통한 상태 일관성 확보
- useEffect() hook을 이용
- axios 사용을 통해 코드 단순화 가능(미사용)

1.6 외부 의존성

- 라이브러리:, `react-router-dom`,`react-redux`
- 스타일: `tailwindcss`

2. Admin

2.1 React 컴포넌트 개요

컴포넌트명

AdminHome

목적 및 역할

- 관리자 역할의 사용자를 위한 메인 페이지 컴포넌트
- 관리자가 가입 승인과 회원 관리와 같은 주요 관리 기능을 한 페이지에서 쉽게 접근하고 사용

주요 기능 및 요구사항:

- 동적 컴포넌트 렌더링: '가입 승인' 및 '회원 관리' 버튼 클릭에 따라 SignupApprove 컴포넌트와 MemberManage 컴포넌트를 동적으로 렌더링합니다.
- 상태 관리: useState 훅을 사용하여 현재 어떤 하위 컴포넌트를 보여줄지 상태를 관리합니다.
- UI/UX: 관리자가 직관적으로 원하는 기능 페이지로 이동할 수 있도록 두 가지 관리 기능에 대한 버튼을 제공합니다.

2.2. 컴포넌트 구조 설계

컴포넌트 분류

- 타입: [✔] 페이지 컴포넌트 [ ] 레이아웃 컴포넌트 [] 기능 컴포넌트 [ ] 공통 컴포넌트
- 복잡도: [ ] 단순 (50줄 이하) [✔] 보통 (50-100줄) [ ] 복합 (100줄 이상)
- 상태 관리: [✔] 로컬 상태만 [] Redux 연결 [ ] 상태 없음

파일 구조

```
src/
├── components/
│   ├── admin/
│   │   ├── AdminHome.tsx
│   │   ├── MemberManage.tsx
│   │   └── SignupApprove.tsx
```

2.3 상태 관리 설계

로컬상태

상태명	타입	초기값	용도
selectedComponent	<ComponentType>	null	component선택

2.4구현 세부사항

주요 컴포넌트 목록

컴포넌트명	위치	역할	재사용성
MemberManage	/components/admin	사용자 관리	보통
SignupApprove	/components/admin	사용자 가입승인	보통

API 연동

API 엔드포인트	메서드	용도	호출시점
/api/admin/users	GET	전체 사용자 조회	항목 선택
/api/admin/users/${userid}	PUT	사용자 정보 수정	버튼 선택
/api/admin/users/${userid}	DELETE	사용자 탈퇴	버튼 선택
/api/admin/users/status/pending	GET	가입승인 필요 사용자 조회	항목 선택
/api/admin/users/${user.id}/approve	PATCH	가입승인	승인 선택
/api/admin/users/${user.id}/deny	PATCH	가입거부	거부 선택

2.5 성능 최적화

- 가벼운 컴포넌트
- 필요시 컴포넌트를 가져옴

3. Teacher

3.1 React 컴포넌트 개요

컴포넌트명

TeacherPage

목적 및 역할

- 강사 역할의 사용자를 위한 메인 페이지 컴포넌트.
- 강사가 담당하는 과제, 공지사항, 학생 제출물 관리 등 모든 주요 기능에 대한 통합된 대시보드와 네비게이션을 제공하는 역할
- 강사 관련 기능의 최상위 진입점 역할 수행

주요 기능 및 요구사항:

- 기능 접근성 제공: 과제 생성, 과제 목록 확인, 공지사항 관리 등 강사에게 필요한 모든 기능으로 쉽게 이동할 수 있는 UI를 제공합니다.
- 동적 컨텐츠 렌더링: 사용자의 선택(예: 과제 관리, 제출물 확인 등)에 따라 Dashboard, AssignmentCreate, AssignmentList, AssignmentReview, GradePage, NoticeCreate, NoticeEdit 등과 같은 하위 컴포넌트를 동적으로 로드하고 렌더링합니다.
- 라우팅 관리: 하위 기능 페이지로의 이동을 처리하며, URL 파라미터를 사용하여 특정 과제나 제출물을 수정하거나 채점하는 등의 기능을 지원합니다.
- 사용자 경험 향상: 강사가 자신의 업무를 효율적으로 관리할 수 있도록 직관적인 인터페이스와 네비게이션 흐름을 제공합니다.

3.2. 컴포넌트 구조 설계

컴포넌트 분류

- 타입: [✔] 페이지 컴포넌트 [ ] 레이아웃 컴포넌트 [] 기능 컴포넌트 [ ] 공통 컴포넌트
- 복잡도: [ ] 단순 (50줄 이하) [ ] 보통 (50-100줄) [✔] 복합 (100줄 이상)
- 상태 관리: [✔] 로컬 상태만 [✔] Redux 연결 [ ] 상태 없음

파일 구조

```
src/
├── pages/                  //페이지 컴포넌트
│   └── TeacherPage.tsx
├── components/             // 기능 컴포넌트
│   ├── teacher/
│   │   ├── AssignmentCreate.tsx
│   │   ├── AssignmentEdit.tsx
│   │   ├── AssignmnetForm.tsx
│   │   ├── AssignmentList.tsx
│   │   ├── AssignmentReview.tsx
│   │   ├── Dashboard.tsx
│   │   ├── GradePage.tsx
│   │   ├── NoticeCreate.tsx
│   │   └── NoticeEdit.tsx
```

```
├── types/
│   └── assignment.ts
└── api.ts
```

3.3 상태 관리 설계

로컬상태

상태명	타입	초기값	용도
currentPage	<PageType>	'dashboard'	nav선택에 따른 화면 동적 렌더링

3.4 구현 세부사항

주요 컴포넌트 목록

컴포넌트명	위치	역할	재사용성
AssignmentCreate	/components/teacher	과제생성	낮음
AssignmentEdit	/components/teacher	과제수정	낮음
AssignmentForm	/components/teacher	페이지표시	낮음
AssignmentList	/components/teacher	목록표시	낮음
AssignmentReview	/components/teacher		낮음
Dashboard	/components/teacher	목록표시	낮음
GradePage	/components/teacher		낮음
NoticeCreate	/components/teacher	공지사항생성	낮음
NoticeEdit	/components/teacher	공지사항수정	낮음
AssignmentCreate	/components/teacher	과제생성	낮음
AssignmentEdit	/components/teacher	과제수정	낮음
AssignmentForm	/components/teacher	페이지표시	낮음

API 연동

API 엔드포인트	메서드	용도	호출시점

/api/assignments	POST	과제 생성	버튼선택
/api/assignments	GET	과제 목록 조회	페이지로드
/api/assignments/${assignmentId}	PATCH	과제수정	버튼선택
/api/assignments/submissions /${submissionId}	PATCH	과제물 채점	버튼선택

3.5 성능 최적화

- useEffect() hook을 이용
- 필요시 컴포넌트를 가져옴

3.6 외부 의존성

- 스타일: `bootstrap css`

4. Alarm

4.1 React 컴포넌트 개요

컴포넌트명

AlarmPage

목적 및 역할

- 사용자의 알림 내역을 조회하고 표시하는 역할을 담당
- API를 통해 알림 목록을 가져온 후, 이를 채팅, 공지, 과제와 같은 유형별로 필터링하여 사용자에게 보여줌

주요 기능 및 요구사항:

- 알림 목록 조회: 컴포넌트 마운트 시 API를 호출하여 사용자의 전체 알림 목록을 가져와야 합니다.
- 유형별 필터링: '전체', '채팅', '공지', '과제' 탭을 제공하여 사용자가 원하는 유형의 알림만 볼 수 있도록 필터링 기능을 구현해야 합니다.
- 알림 항목 표시: 각 알림은 내용, 생성 일시, 관련 페이지로 이동하는 링크 등 필요한 정보를 포함해야 합니다.
- 로딩 및 오류 처리: API 호출 중에는 로딩 상태를, 호출 실패 시에는 사용자에게 오류 메시지를 표시해야 합니다.
- 빈 상태 처리: 알림이 없을 경우, 사용자에게 "새로운 알림이 없습니다."와 같은 메시지를 표시해야 합니다.

4.2. 컴포넌트 구조 설계

컴포넌트 분류

- 타입: [✔] 페이지 컴포넌트 [ ] 레이아웃 컴포넌트 [] 기능 컴포넌트 [ ] 공통 컴포넌트

- 복잡도: [ ] 단순 (50줄 이하) [ ] 보통 (50-100줄) [✔] 복합 (100줄 이상)
- 상태 관리: [✔] 로컬 상태만 [✔] Redux 연결 [ ] 상태 없음

파일 구조

```
src/
├── pages/
    └── AlarmPage.tsx
```

4.3 상태 관리 설계

로컬상태

상태명	타입	초기값	용도
activeTab	string	'all'	현재 선택된 알림 필터 탭 관리
histories	History[]	[]	API 호출을 통해 가져온 알림 내역 목록 저장
loading	boolean	true	알림 데이터를 불러오는 동안 로딩 상태를 표시하는 데 사용
error	string | null	null	API 호출 실패 시 발생한 오류 메시지를 저장

Redux상태

액션타입	Payload	설명
state.auth.user.token	없음	API 호출 시 사용자 인증을 위해 토큰을 가져오는 데 사용

4.4 구현 세부사항

API 연동

API 엔드포인트	메서드	용도	호출시점
/api/histories	GET	알림 목록 조회	페이지로

			드

4.5 성능 최적화

- Redux를 통한 상태 일관성 확보
- useEffect() hook을 이용하여 컴포넌트 마운트 시 api 호출
- axios 사용을 통해 코드 단순화 가능(미사용)

4.6 외부 의존성

- 라이브러리: `react-router-dom, react-redux`
- 스타일: `bootstrap css`

5. Assignment

5.1 React 컴포넌트 개요

컴포넌트명

AssignmentPage

목적 및 역할

- 사용자의 과제 목록을 불러와 표시하고, 특정 과제를 선택했을 때 그 상세 내용을 보여줌
- 사용자가 과제를 파일 또는 링크 형태로 제출하고 제출 내역 및 피드백을 확인할 수 있도록 하는 핵심적인 역할 수행

주요 기능 및 요구사항:

- 과제 목록 조회 및 선택: 컴포넌트 마운트 시 모든 과제 목록을 API를 통해 가져오고, 사용자가 목록에서 과제를 선택하면 해당 과제의 상세 정보가 표시되어야 합니다.
- 과제 상세 정보 표시: 선택된 과제의 제목, 마감일, 상태(진행 중/완료), 설명 등을 사용자에게 보여줘야 합니다.
- 과제 제출 기능: 사용자가 파일 업로드 또는 링크 입력 방식 중 하나를 선택하여 과제를 제출할 수 있는 인터페이스를 제공해야 합니다.
- 제출 상태 관리: 제출이 진행 중일 때 로딩 상태를 표시하고, 성공 또는 실패에 대한 알림을 사용자에게 제공해야 합니다.
- 제출 내역 및 피드백 탭: 사용자가 자신의 과제 제출 기록과 교사의 피드백을 확인할 수 있는 탭 인터페이스를 포함해야 합니다.
- 로딩 및 오류 처리: 과제 목록을 불러오거나 제출하는 동안 로딩 상태를 표시하고, API 호출 실패 시 오류 메시지를 사용자에게 알려야 합니다.

5.2. 컴포넌트 구조 설계

컴포넌트 분류

- 타입: [✔] 페이지 컴포넌트 [ ] 레이아웃 컴포넌트 [✔] 기능 컴포넌트 [ ] 공통 컴포넌트
- 복잡도: [ ] 단순 (50줄 이하) [ ] 보통 (50-100줄) [✔] 복합 (100줄 이상)
- 상태 관리: [✔] 로컬 상태만 [✔] Redux 연결 [ ] 상태 없음

파일 구조

```
src/
├── pages/
    └── AlarmPage.tsx
```

5.3 상태 관리 설계

로컬상태

상태명	타입	초기값	용도
assignments	Assignment[]	[]	API를 통해 가져온 전체 과제 목록을 저장
selectedAssignment	Assignment | null	null	현재 사용자가 선택한 과제의 상세 정보를 저장
loading	boolean	true	과제 목록을 불러오는 동안 로딩 상태를 표시
error	string | null	null	API 호출 실패 시 발생한 오류 메시지를 저장
activeTab	string	‘description’	현재 활성화된 과제 상세 정보 탭을 관리
submissionType	'FILE' | 'LINK'	'LINK'	사용자가 선택한 과제 제출 방식(파일 또는 링크)을 관리
file	File | null	null	파일 제출 시 사용자가 선택한 파일 객체를 저장
linkUrl	string	' '	링크 제출 시 사용자가 입력한 URL을 저장

submitting	boolean	false	과제 제출 API 호출 중인지 여부를 나타냄
submitSuccess	boolean	false	과제 제출 성공 여부를 나타

Redux상태

액션타입	Payload	설명
state.auth.user.token	없음	API 호출 시 사용자 인증을 위해 토큰을 가져오는 데 사용

5.4 구현 세부사항

API 연동(추가 필요)

API 엔드포인트	메서드	용도	호출시점
/api/assignments	GET	과제 목록 조회	페이지로드
/api/assignments/${assignmentId}	GET	과제 세부사항 조회	페이지로드
/api/assignments/${assignmentId}/submissions	POST	과제 제출	버튼 클릭

5.5 성능 최적화

- Redux를 통한 상태 일관성 확보
- useEffect() hook을 이용하여 컴포넌트 마운트 시 api 호출

5.6 외부 의존성

- 라이브러리: `react-router-dom, react-redux`
- 스타일: `bootstrap 유틸리티 사용`

6. chatroom

6.1 React 컴포넌트 개요

컴포넌트명

ChatRoomPage

목적 및 역할

- 사용자가 특정 채팅방에 참여하여 실시간으로 메시지를 주고받는 기능을 제공

주요 기능 및 요구사항:

- 채팅방 입장 및 퇴장 관리: 컴포넌트 마운트 시 소켓 서비스를 통해 채팅방에 입장하고, 언마운트 시 퇴장합니다.
- 실시간 메시지 수신: 웹소켓을 통해 서버로부터 새로운 메시지를 실시간으로 수신하여 화면에 표시합니다.
- 메시지 전송: 사용자가 입력한 텍스트를 웹소켓을 통해 서버로 전송합니다.
- 참여자 목록 표시: 채팅방에 참여하고 있는 다른 사용자 목록을 표시합니다.
- 스크롤 자동 이동: 새 메시지가 추가될 때마다 자동으로 스크롤을 최하단으로 이동시켜 최신 메시지가 보이도록 합니다.
- 로딩 및 오류 처리: 메시지 및 참여자 목록을 불러오는 동안 로딩 상태를 표시하고, 오류 발생 시 사용자에게 메시지를 보여줍니다.

6.2. 컴포넌트 구조 설계

컴포넌트 분류

- 타입: [✔] 페이지 컴포넌트 [ ] 레이아웃 컴포넌트 [✔] 기능 컴포넌트 [ ] 공통 컴포넌트
- 복잡도: [ ] 단순 (50줄 이하) [ ] 보통 (50-100줄) [✔] 복합 (100줄 이상)
- 상태 관리: [✔] 로컬 상태만 [✔] Redux 연결 [ ] 상태 없음

파일 구조

```
src/
├── pages/
│     ├── ChatRoomPage.tsx
├── components/
│     ├── admin/
│     ├── MessageBubble.tsx
│     ├── MessageInput.tsx
│     ├── ParticipantList.tsx
│     └── UserSelectionModel.tsx
├── services/
│     └── socketService.ts
├── store/
│     ├──slices/
│     │      └── authSlice.ts
│     ├── index.ts
│     └── hooks.ts

├── types/
│     ├── assignment.ts
│     └── index.ts
```

└── [api.ts]

6.3 상태 관리 설계

로컬상태

상태명	타입	초기값	용도
messages	ChatMessage[]	[]	채팅방 메시지 목록을 저장
participants	ChatRoomParticipant[]	[]	채팅방 참여자 목록을 저장.
loading	boolean	false	컴포넌트가 처음 로딩 중인지 여부를 나타냄
messagesLoading	boolean	true	메시지 데이터를 로딩 중인지 여부를 나타냄
roomType	'ONE_TO_ONE' | 'GROUP'	'GROUP'	채팅방의 타입을 나타냄
roomName	string	''	채팅방의 이름을 저장

Redux상태

액션타입	Payload	설명
setUserData	object	로그인 성공 후 백엔드로부터 받은 사용자 데이터를 전역 상태에 저장
logout	void	사용자 인증 상태를 초기화하여 로그아웃 처리
initializeAuth	void	앱 시작 시 로컬/세션 저장소에서 사용자 정보를 복원 또는 세션 스토리지에 저장된 토큰을 확인하여 인증 상태를 초기화
setEmail	string	비밀번호 찾기 과정에서 사용자가 입력한 이메일을 임시로 저장

setVerifyCode	string	비밀번호 찾기 과정에서 사용자가 입력한 인증 코드를 저장
setNewPassword	string	새로운 비밀번호 설정 과정에서 사용자가 입력한 새 비밀번호를 저장
resetAuthState	없음	비밀번호 찾기와 같은 특정 인증 관련 프로세스가 완료된 후, 해당 프로세스에 사용된 상태들을 초기화

6.4 구현 세부사항

주요 컴포넌트 목록(추가 수정 필요)

컴포넌트명	위치	역할	재사용성
ParticipantList	/components/	참가자조회	높음
MessageBubble	/components/	발신자별 ui 차별	높음
MessageInput	/components/	메세지 입력	높음

API 연동(추가 수정 필요)

API 엔드포인트	메서드	용도	호출시점
/api/chatrooms/all	GET	채팅방 목록	페이지 로드
/api/chatrooms	POST	채팅방 만들기	버튼선택
/api/chatrooms/${roomId}/join	POST	채팅방 참여	항목선택
/api/chatrooms/${roomId}`	DELETE	채팅방삭제	버튼선택
/api/chatrooms/${roomId}/leave	PUT	채팅방떠나기	버튼선택
/api/chatrooms/${roomId}/participants	GET	채팅방참가자 조회	페이지 로드

6.5 성능 최적화

- Redux를 통한 상태 일관성 확보
- useEffect() hook을 이용

6.6 외부 의존성

- 라이브러리: `react-router-dom, react-redux, axios, websocket`
- 스타일: `bootstrap 유틸리티 사용`

7. HomePage

7.1 React 컴포넌트 개요

컴포넌트명

HomePage

목적 및 역할

- 사용자가 채팅 애플리케이션에 접속했을 때 가장 먼저 보게 되는 메인 페이지
- 사용자에게 채팅방 목록을 보여주고, 새로운 채팅방을 만들거나 기존 채팅방에 참여할 수 있는 인터페이스를 제공하는 것

주요 기능 및 요구사항:

- 채팅방 목록 조회 및 선택: 컴포넌트가 마운트될 때 모든 채팅방 목록을 API를 통해 가져와 표시해야 합니다. 사용자는 목록에서 채팅방을 선택하여 해당 방으로 이동할 수 있습니다.
- 채팅방 생성 기능: 사용자가 새로운 그룹 채팅방 또는 1:1 채팅방을 만들 수 있는 인터페이스를 제공해야 합니다. 1:1 채팅방 생성 시에는 상대방을 선택할 수 있는 모달을 표시해야 합니다.
- 채팅방 관리: 사용자가 목록에서 특정 채팅방을 나갈 수 있는 기능을 제공해야 합니다.
- 채팅방 검색 및 필터링: 사용자가 검색어를 입력하면 채팅방 이름을 기준으로 목록을 실시간으로 필터링하여 보여줘야 합니다.
- 상태 관리 및 내비게이션: 채팅방 목록, 검색 상태, 로딩 상태 등을 효율적으로 관리해야 하며, '공지사항' 및 '과제' 페이지로 이동하는 내비게이션 기능을 포함해야 합니다.
- 로딩 및 오류 처리: 채팅방 목록을 불러오거나 채팅방을 생성, 참여, 나가는 동안 로딩 상태를 표시해야 합니다. API 호출 실패 시, 구체적인 오류 메시지를 사용자에게 알림으로 제공해야 합니다.

7.2. 컴포넌트 구조 설계

컴포넌트 분류

- 타입: [ ] 페이지 컴포넌트 [ ] 레이아웃 컴포넌트 [✔] 기능 컴포넌트 [ ] 공통 컴포넌트
- 복잡도: [ ] 단순 (50줄 이하) [ ] 보통 (50-100줄) [✔] 복합 (100줄 이상)
- 상태 관리: [✔] 로컬 상태만 [✔] Redux 연결 [ ] 상태 없음

파일 구조

```
src/
├── pages/                    //페이지 컴포넌트
│     ├── HomePage.tsx
└── api.ts
```

7.3 상태 관리 설계

로컬상태

상태명	타입	초기값	용도
rooms	ChatRoom[]	[]	서버로부터 받아온 채팅방 목록을 저장
newRoomName	string	''	새로운 채팅방을 생성하기 위해 입력된 이름을 저장
newRoomType	`ChatRoomType`	'GROUP'	새로운 채팅방의 유형(`'GROUP'` 또는 `'ONE_TO_ONE'`)을 저장합니다.
loading	`boolean`	false	API 호출이 진행 중인지 여부를 나타내 로딩 UI를 표시
searchTerm	`string`	`''`	채팅방 검색 필드에 입력된 문자열을 저장
showSearch	`boolean`	`false`	검색창의 표시 여부를 제어
showUserSelection Modal	`boolean`	`false`	1:1 채팅방 생성 시 상대방 선택 모달의 표시 여부를 제어
selectedUser	`User | null`	`null`	1:1 채팅방 생성을 위해 선택된 사용자의 정보를

			저장
roomLastMessageTimes	`Record<number, string>`	`{}`	각 채팅방의 마지막 메시지 시간을 저장하며, 로컬 스토리지에 유지

Redux상태

액션타입	Payload	설명
setUserData	object	로그인 성공 후 백엔드로부터 받은 사용자 데이터를 전역 상태에 저장
logout	없음	사용자 인증 상태를 초기화하여 로그아웃 처리
initializeAuth	없음	애플리케이션 시작 시 로컬 또는 세션 스토리지에 저장된 토큰을 확인하여 인증 상태를 초기화
setEmail	string	비밀번호 찾기 과정에서 사용자가 입력한 이메일을 임시로 저장
setVerifyCode	string	비밀번호 찾기 과정에서 사용자가 입력한 인증 코드를 저장
setNewPassword	string	새로운 비밀번호 설정 과정에서 사용자가 입력한 새 비밀번호를 저장
resetAuthState	없음	비밀번호 찾기와 같은 특정 인증 관련 프로세스가 완료된 후, 해당 프로세스에 사용된 상태들을 초기화

7.4 구현 세부사항

API 연동

API 엔드포인트	메서드	용도	호출시점

/api/chatrooms	POST	채팅룸만들기	버튼 선택
/api/chatrooms/${roomId}/join	POST	채팅방 참여	항목선택

7.5 성능 최적화

- Redux를 통한 상태 일관성 확보
- useEffect() hook을 이용

7.6 외부 의존성

- 라이브러리: `react-router-dom, react-redux, axios, websocket`
- 스타일: `bootstrap 유틸리티 사용`

8. NoticePage

8.1 React 컴포넌트 개요

컴포넌트명

NoticePage

목적 및 역할

- 사용자에게 공지사항을 표시하고 관리하기 위한 전용 컴포넌트
- 공지사항 목록을 불러와 보여주는 것이며, 선생님이나 관리자 권한을 가진 사용자는 새로운 공지사항을 작성, 수정, 삭제할 수 있도록 하는 핵심적인 역할을 수행

주요 기능 및 요구사항:

- 공지사항 목록 조회 및 선택: 컴포넌트가 마운트될 때 모든 공지사항 목록을 API를 통해 가져와 표시해야 합니다. 사용자는 목록에서 공지사항을 선택하여 그 상세 내용을 모달 창으로 확인할 수 있어야 합니다.
- 공지사항 CRUD 기능: 선생님 또는 관리자 권한을 가진 사용자에게는 공지사항을 작성(Create), 수정(Update), 삭제(Delete) 할 수 있는 기능을 제공해야 합니다.
- 공지사항 상세 정보 표시: 공지사항 목록에서 항목을 클릭하면 제목, 내용, 작성자, 작성일 등 상세 정보가 포함된 모달이 나타나야 합니다.
- 페이지네이션 기능: API 응답에 포함된 페이지 정보(`currentPage`, `totalPages`)를 기반으로 페이지 이동 기능을 제공해야 합니다.
- 부수적 기능: 중요 공지사항(pinned)은 특별한 뱃지(Badge)로 구분하여 강조해야 하고, 공지사항 목록의 통계(전체, 중요, 일반)를 제공하며, 최근 공지사항 3개를 별도로 표시해야 합니다.

8.2. 컴포넌트 구조 설계

컴포넌트 분류

- 타입: [✔] 페이지 컴포넌트 [ ] 레이아웃 컴포넌트 [✔] 기능 컴포넌트 [ ] 공통 컴포넌트
- 복잡도: [ ] 단순 (50줄 이하) [ ] 보통 (50-100줄) [✔] 복합 (100줄 이상)
- 상태 관리: [✔] 로컬 상태만 [✔] Redux 연결 [ ] 상태 없음

파일 구조
파일 구조

```
src/
├── pages/                    //페이지 컴포넌트
│     ├── NoticePage.tsx
└── api.ts
```

8.3 상태 관리 설계

로컬상태

상태명	타입	초기값	용도
notices	Notice[]	[]	서버에서 불러온 공지사항 목록을 저장
loading	`boolean`	true	API 호출이 진행 중인지 나타
error	`string	null`	null	API 호출 실패 시 에러 메시지를 저장
selectedNotice	`Notice	null`	`null`	사용자가 선택하여 상세 정보를 볼 공지사항 객체를 저장
showModal	`boolean`	`false`	공지사항 상세 보기 모달의 표시 여부를 제어
showSearch	`boolean`	`false`	검색창의 표시 여부를 제어
showCreateModal	`boolean`	`false`	공지사항 작성/수정 모달의 표시 여부를 제어
isEditing	`boolean`	`false`	현재 상태가 수정 모드인지(기존 공지 수정) 여부를 나타
formData	`{ title: string, content: string, pinned: boolean }`	`{ title: '', content: '', pinned: false }`	공지사항 작성/수정 시 폼 데이터를 임시로 저장

currentPage	number	0	현재 페이지 번호를 저장
totalPages	number	1	전체 페이지 수를 저장
totalElements	number	2	전체 공지사항 수를 저장

Redux상태

액션타입	Payload	설명
setUser	{ user: User, token: string }	사용자가 로그인에 성공했을 때, 사용자 정보와 토큰을 스토어에 저장
logout	없음	사용자 인증 상태를 초기화하여 로그아웃 처리

8.4 구현 세부사항

API 연동

API 엔드포인트	메서드	용도	호출시점
/api//announcements?page=${page}&size=${size}&sort=${sort}	GET	공지사항 목록조회	페이지로드
/api/announcements/${noticeId}	GET	특정공지 조회	버튼클릭

8.5 성능 최적화

- useEffect() hook을 이용

8.6 외부 의존성

- 라이브러리: `react-router-dom`, `axios`
- 스타일: `bootstrap` 유틸리티 사용

9. ProfilePage

9.1 React 컴포넌트 개요

컴포넌트명

ProfilePage

목적 및 역할

- 사용자가 자신의 프로필 정보를 관리하고 업데이트할 수 있는 인터페이스를 제공
- 사용자는 이 페이지에서 자신의 이름, 이메일, 역할과 같은 기본 정보를 확인하고, 프로필 사진을 변경하거나 이름 및 비밀번호를 수정 가능.

주요 기능 및 요구사항:

- 프로필 정보 표시: 현재 로그인된 사용자의 이름(`fullName` 또는 `username`), 이메일, 역할을 화면에 표시해야 합니다.
- 아바타 관리: `AvatarUploader` 컴포넌트를 통해 사용자가 자신의 프로필 사진을 업로드하고 변경할 수 있어야 합니다.
- 프로필 수정: 사용자는 '이름 변경' 및 '비밀번호 변경' 버튼을 클릭하여 이름 또는 비밀번호를 수정할 수 있는 모달 창을 열 수 있어야 합니다.
- 계정 삭제 기능: 계정을 삭제하는 기능을 위한 버튼을 제공해야 합니다.
- 모달 관리: 사용자의 수정 요청(`name` 또는 `password`)에 따라 적절한 유형의 `ProfileEditModal`을 표시해야 합니다.

9.2. 컴포넌트 구조 설계

컴포넌트 분류

- 타입: [✔] 페이지 컴포넌트 [ ] 레이아웃 컴포넌트 [✔] 기능 컴포넌트 [ ] 공통 컴포넌트
- 복잡도: [ ] 단순 (50줄 이하) [ ] 보통 (50-100줄) [✔] 복합 (100줄 이상)
- 상태 관리: [✔] 로컬 상태만 [✔] Redux 연결 [ ] 상태 없음

파일 구조

```
src/
├── pages/                  //페이지 컴포넌트
│     ├── ProfilePage.tsx
├── components/             // 기능 컴포넌트
│     ├── profile/
                └── AvatarUploader.tsx
└── api.ts
```

9.3 상태 관리 설계

로컬상태

상태명	타입	초기값	용도
showModal	boolean	false	프로필 수정 모달의 표시 여부를 제어
modalType	`'name' | 'password'`	`'name'`	어떤 유형의 수정(이름 또는 비밀번호)을 위한 모달인지 결정

Redux상태

액션타입	Payload	설명
`setUser`	{ user: User, token: string }	사용자가 로그인에 성공했을 때, 사용자 정보와 토큰을 스토어에 저장
logout	없음	사용자 인증 상태를 초기화하여 로그아웃 처리

9.4 구현 세부사항

주요 컴포넌트 목록

컴포넌트명	위치	역할	재사용성

API 연동

API 엔드포인트	메서드	용도	호출시점

/api/users/${userId}/avatar	POST	프로필 사진 변경	버튼선택

9.5 성능 최적화

- Redux를 통한 상태 일관성 확보
- useEffect() hook을 이용

9.6 외부 의존성

- 라이브러리: `react-redux, axios`
- 스타일: `bootstrap` 유틸리티 사용